

TOPIC 4: SOFTWARE TESTING FUNDAMENTALS

A **software bug** is any imperfection in the code that causes a program to produce incorrect results, crash unexpectedly, or behave in unintended ways. Bugs occur due to human errors during requirements analysis, design, coding, or configuration. Unlike hardware faults that wear out over time, software bugs exist from the moment faulty code is written and remain until deliberately corrected.

In practice, a bug represents a mismatch between what the software does and what it should do according to specifications. Every software failure traces back to a bug—whether the program divides by zero, miscalculates totals, or ignores user input validation. Understanding bugs forms the foundation of testing because testers don't create bugs; they discover and document them for developers to fix.

The cost of bugs escalates dramatically through the development lifecycle. A logic error caught during code review costs less money to fix, but the same error discovered after deployment in production costs much more due to customer support, lost revenue, legal liability, and emergency patches. This economic reality drives systematic testing practices.

MAIN TYPES OF BUGS

FUNCTIONAL BUGS

Functional bugs occur when individual features or components fail to perform their specified operations correctly. These defects manifest as buttons that don't respond, forms that submit empty data, or calculations that produce wrong outputs. The software technically runs but delivers incorrect functionality, breaking the core user workflows developers promised to build.

PERFORMANCE BUGS

Performance bugs degrade system responsiveness, throughput, or resource efficiency under normal or peak workloads. These defects appear as slow page loads, memory leaks that consume RAM over time, database queries taking seconds instead of milliseconds, or applications freezing during concurrent user access. While functionally correct, the software fails quality standards for speed and scalability expected in real-world deployment.

USABILITY BUGS

Usability bugs create software that works correctly but confuses or frustrates users through poor interface design, unclear navigation, inconsistent workflows, or hidden critical functions. These defects stem from inadequate user experience research rather than programming errors. Users complete tasks correctly but waste excessive time or abandon the application entirely due to cognitive overload.

SECURITY BUGS

Security bugs expose systems to unauthorized access, data breaches, privilege escalation, or information leakage through vulnerabilities like SQL injection, cross-site scripting, weak encryption, or improper authentication. Attackers exploit these flaws systematically, compromising entire systems rather than causing random failures. Security defects represent the highest business risk due to regulatory fines, legal liability, and permanent reputation damage.

COMPATIBILITY BUGS

Compatibility bugs cause software to malfunction across different hardware configurations, operating systems, browsers, screen resolutions, or network conditions. The same code executes perfectly on developer machines but crashes, renders incorrectly, or loses functionality on customer devices. These defects arise from untested platform variations and assumptions about uniform execution environments.

LOGIC BUGS

Logic bugs produce correct syntax execution but wrong computational results due to flawed algorithms, incorrect conditional branching, boundary condition oversights, or business rule misinterpretations. The software runs without errors or crashes yet delivers incorrect decisions, calculations, or data processing. These prove hardest to detect because no obvious failure symptoms appear—only wrong answers.

BUG LIFECYCLE (DEFECT MANAGEMENT PROCESS)

The bug lifecycle provides standardized workflow tracking every defect from discovery through permanent resolution. Development teams use tools like Jira, Bugzilla, or Azure DevOps to automate status transitions and maintain audit trails.

NEW: Tester documents defect with precise reproduction steps, screenshots, logs, expected vs. actual results, and environment details. Development team receives complete information for diagnosis.

OPEN: Triage team validates reported issue constitutes genuine software defect rather than user error, misconfiguration, or duplicate report. Invalid reports receive rejection with explanation.

ASSIGNED: Technical lead allocates defect to specific developer based on code ownership, expertise, and current workload capacity. Developer receives clear priority ranking and target resolution date.

IN PROGRESS: Assigned developer analyzes root cause, implements correction, conducts local verification testing, and commits fix to source control. Developer updates status with progress notes and estimated completion.

FIXED: Developer declares resolution complete and deploys corrected code into testing environment. Build system generates new version containing all accumulated fixes for verification testing.

RETEST: Original tester or designated verifier re-executes original failing test cases against fixed version. Tester documents whether fix resolves issue completely or introduces regressions.

VERIFIED: Independent quality assurance confirms fix eliminates original defect without creating new failures in related functionality. Final acceptance criteria met across all test scenarios.

CLOSED: Defect formally archived with resolution summary, root cause analysis, and lessons learned. Historical data becomes available for metrics analysis and process improvement.

REOPENED: Fix proves inadequate or introduces new related failures. Defect returns to ASSIGNED status with updated reproduction steps and severity reassessment for second pass correction.

This structured process ensures no defects disappear into untracked limbo while providing management visibility into testing progress, developer productivity, and overall software quality trends.